

XGBoost for Regression

1) High-Level Idea

XGBoost builds an ensemble of regression trees **additively**, where each tree is fit to reduce the current loss using **gradients** (and Hessians), and the whole system adds regularization to avoid overfitting.

2) The Objective (What XGBoost Minimizes)

For a dataset with targets y_i and model prediction $\hat{y}_i$, the objective after adding T trees is:

$$\text{Obj} = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{t=1}^T \Omega(f_t)$$

where each tree f_t contributes a regularization term (complexity penalty):

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^J w_j^2 + \alpha \sum_{j=1}^J |w_j|$$

- $l(\cdot, \cdot)$ = loss (for regression usually squared error or other)
 - λ = L2 regularization on leaf weights
 - α = L1 regularization (optional)
 - γ = penalty per leaf (minimum gain to create a leaf)
-

3) Additive / Stagewise Model

The model is built in rounds:

$$\hat{y}_i^{(0)} \text{ (init)} \rightarrow \hat{y}_i^{(m)} = \hat{y}_i^{(m-1)} + \eta f_m(x_i)$$

- f_m is the new tree, η is the learning rate (shrinkage).
-

4) Use of Gradients & Hessians

At round m we compute first and second derivatives of the loss w.r.t. current prediction:

$$g_i = \left. \frac{\partial l(y_i, \hat{y}_i)}{\partial \hat{y}_i} \right|_{\hat{y}_i = \hat{y}_i^{(m-1)}}, \quad h_i = \left. \frac{\partial^2 l(y_i, \hat{y}_i)}{\partial \hat{y}_i^2} \right|_{\hat{y}_i = \hat{y}_i^{(m-1)}}$$

These are used as *pseudo-responses* to fit the next tree.

- For squared error $l = \frac{1}{2}(y - \hat{y})^2$: $g_i = \hat{y}_i - y_i$, $h_i = 1$.

5) How a Tree is Scored (Optimal Leaf Weight)

Suppose a leaf j contains a set of samples I_j . Define:

$$G_j = \sum_{i \in I_j} g_i, \quad H_j = \sum_{i \in I_j} h_i.$$

If we assign that leaf a weight w_j , the optimal weight (closed form) minimizing the regularized quadratic approximation is:

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

The contribution (gain) of having that leaf (versus not having it) is proportional to:

$$\text{LeafScore}_j = \frac{1}{2} \cdot \frac{G_j^2}{H_j + \lambda} \quad (\text{up to constants and } \gamma)$$

6) Split Gain (How XGBoost Picks Best Split)

When testing a candidate split that creates left node L and right node R from parent P :

$$G_P = G_L + G_R, \quad H_P = H_L + H_R$$

The gain from making the split is:

$$\text{Gain} = \frac{1}{2} \left(\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G_P^2}{H_P + \lambda} \right) - \gamma$$

- XGBoost evaluates this for many candidate splits and picks the split with the highest positive Gain (and `> gamma`).
- `min_child_weight` is a threshold on H (sum of h) in a child to prevent splits with too few supporting weight.

7) Tree Construction (Greedy)

1. Compute g_i, h_i for current round.
2. For each feature, try split points; compute G_L, H_L, G_R, H_R and Gain.
3. Choose the best split (Gain maximal and $> \text{gamma}$).
4. Recurse until `max_depth`, `min_child_weight`, or no positive gain.
5. For each leaf compute w_j^* and the leaf prediction is $\eta \cdot w_j^*$ (added to model).

XGBoost grows trees with greedy search and prunes (backwards) based on Gain.

8) Prediction Update & Learning Rate

After fitting the tree, update:

$$\hat{y}_i^{(m)} = \hat{y}_i^{(m-1)} + \eta \cdot w_{leaf(i)}^*$$

- η (learning rate) scales contributions. Smaller $\eta \rightarrow$ need more trees but often better generalization.
-

9) Numerical Mini-Example (Step-by-Step, Simple Numbers)

Small toy: 3 samples with targets $[3, 5, 7]$. Start with $\hat{y}^{(0)} = 0$ for all. Use squared error loss; set $\lambda = 0, \gamma = 0, \eta = 0.1$ to keep numbers simple.

- Gradients $g_i = \hat{y} - y = 0 - y = -y$. So $g = [-3, -5, -7]$. Hessians $h = [1, 1, 1]$.
- Total at root: $G_P = -15, H_P = 3$.

No split (single leaf):

- $w_{root}^* = -G_P / (H_P + \lambda) = 15/3 = 5$.
- Leaf score = $\frac{1}{2} \cdot \frac{(-15)^2}{3} = \frac{1}{2} \cdot 75 = 37.5$ (this is objective reduction term — absolute values are not critical now).

Try splitting into $L=\{3\}$ and $R=\{5,7\}$:

- $G_L = -3, H_L = 1 \Rightarrow w_L^* = 3$.
- $G_R = -12, H_R = 2 \Rightarrow w_R^* = 6$.
- Gain = $\frac{1}{2} \left(\frac{9}{1} + \frac{144}{2} - \frac{225}{3} \right) = \frac{1}{2} (9 + 72 - 75) = \frac{1}{2} (6) = 3 > 0$, so split is beneficial here (with $\lambda=0$).

- After this split, predictions (one tree) become: left sample gets $\eta \cdot w_L = 0.1 \times 3 = 0.3$; right samples get $0.1 \times 6 = 0.6$.

So one tree already moves predictions toward the targets: sample1: $0 \rightarrow 0.3$; sample2: $0 \rightarrow 0.6$; sample3: $0 \rightarrow 0.6$. Next round recompute gradients and repeat.

Takeaway from example: **G** and **H** drive both the best split decision and the magnitude of leaf weights; η controls how aggressively each tree updates predictions.

10) Important Hyperparameters & What They Change

- eta (learning_rate): shrink leaf weights; smaller $\rightarrow$ safer but needs more trees.
- n_estimators: number of boosting rounds (trees).
- max_depth: depth of each tree (controls complexity).
- min_child_weight: minimum sum of h required in a child (prevents overfitting).
- subsample, colsample_bytree: data/feature subsampling to reduce correlation and variance.
- gamma: minimum split loss (must be exceeded to make a split).
- lambda, alpha: L2/L1 leaf weight regularization.

11) Why XGBoost Is Fast & Effective

- Uses gradient + second-order (Hessian) info for better tree fit.
- Adds explicit regularization of leaf weights.
- Efficient data structure (DMatrix), histogram-based and approximate splits, parallelization, and sparsity-aware algorithms.

12) Practical Tips

- Use small eta (0.01 - 0.2) + larger n_estimators.
- Use early_stopping_rounds on a validation set.
- Tune max_depth, min_child_weight, subsample, colsample_bytree.
- Regularize (lambda, alpha) if overfitting.
- Use feature importance and SHAP to interpret.

13) Quick Memory Hooks

- **G & H** = sums of gradients & Hessians per node $\rightarrow$ drive split decisions and leaf weights.
- **Leaf weight** $w^* = -G / (H + \lambda)$.
- **Split Gain** $= \frac{1}{2}(\text{LeftScore} + \text{RightScore} - \text{ParentScore}) - \gamma$.
- **Update** predictions by adding $\eta \times \text{leaf weight}$.